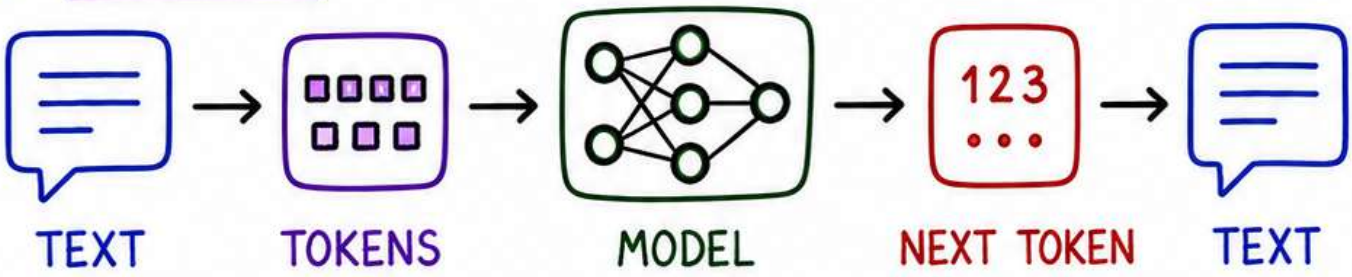
LLM FUNDAMENTALS

★ OVERVIEW

An LLM is a neural network trained to understand and generate text by predicting the next token.

★ CORE FLOW



★ KEY CONCEPTS

- **Token:** Small unit of text processed by the model.
- **Parameters:** Learned values that store model knowledge.
- **Context Window:** Maximum number of tokens the model can process in one request.

★ REMEMBER

LLM = Token Prediction at Scale

– Ketan Sagare

TRANSFORMER ARCHITECTURE

★ OVERVIEW

Transformers are the backbone of modern LLMs.

★ MAIN COMPONENTS

① TOKENIZATION

Text is split into tokens.



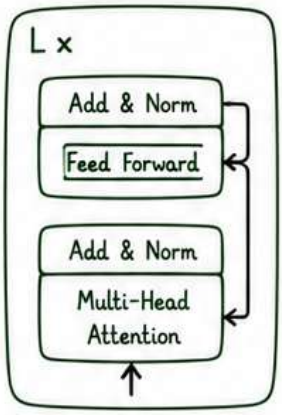
② EMBEDDING

Tokens are converted into vectors.



③ TRANSFORMER BLOCKS

Stack of layers with attention and feed forward.

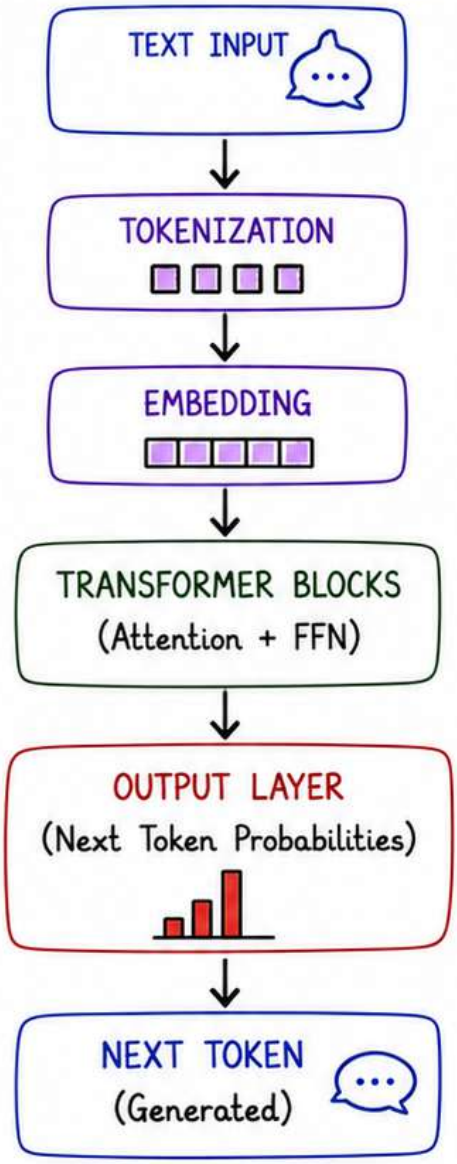


④ OUTPUT LAYER

Converts hidden states to next-token probabilities.



★ ARCHITECTURE FLOW



★ REMEMBER



Transformers process all tokens together using attention, which captures relationships between tokens.

– Ketan Sagare

ATTENTION MECHANISM

★ OVERVIEW

Attention helps the model focus on the most relevant tokens while generating each token.

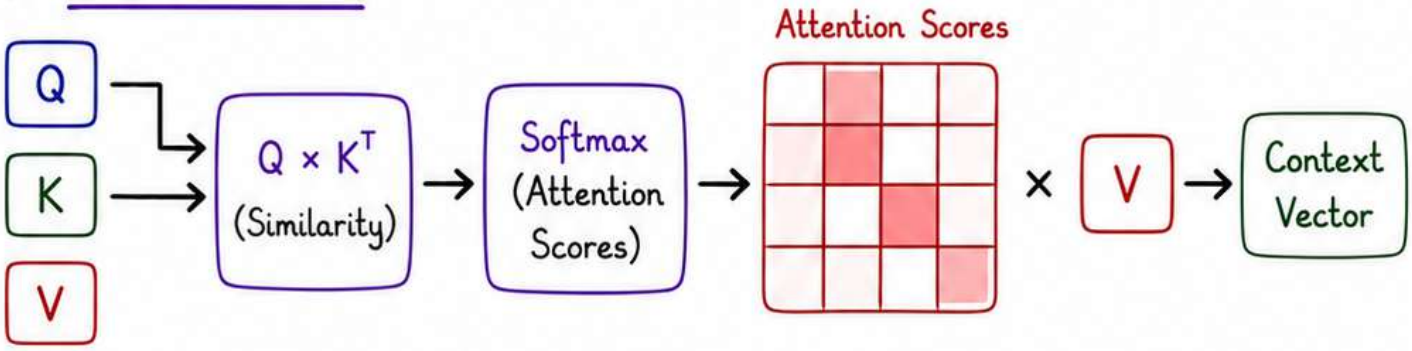
★ Q • K • V

QUERY (Q)
What am I looking for?

KEY (K)
What information do I contain?

VALUE (V)
What information should I provide?

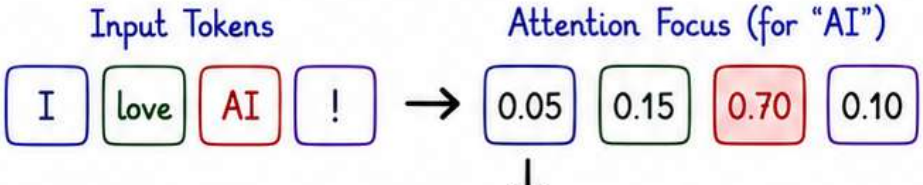
★ HOW IT WORKS



★ BENEFITS

- Captures relationships between tokens
- Handles long-range dependencies
- Allows parallel processing during training

★ VISUAL EXAMPLE



Model pays more attention to "AI" when generating the next token.

★ REMEMBER



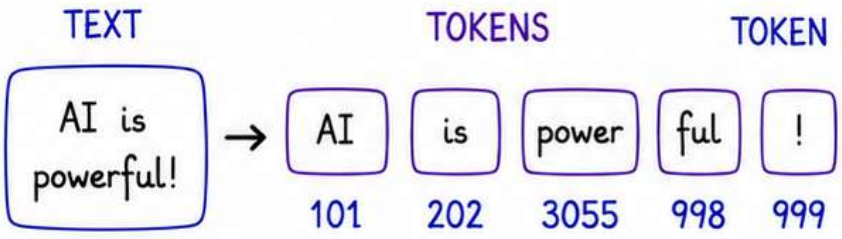
Attention = Find what matters, Focus on it, and use it.

– Ketan Sagare

TOKENIZATION & EMBEDDINGS

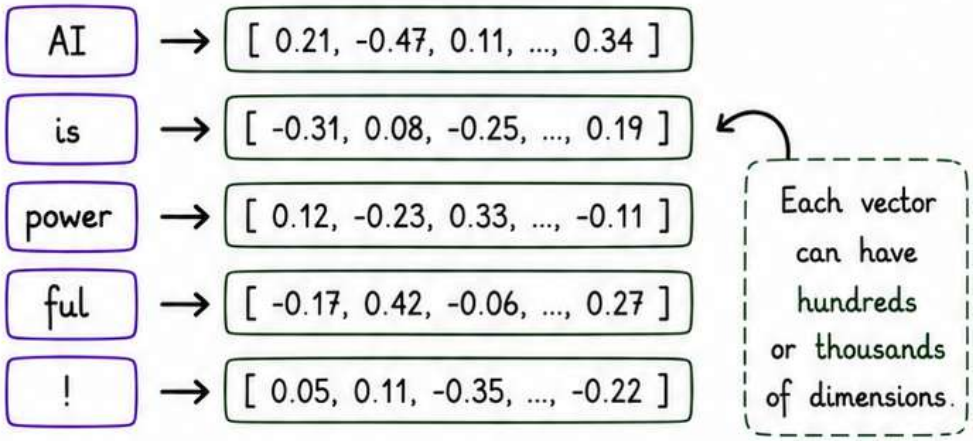
★ 1) TOKENIZATION

Text is split into smaller units called tokens.



★ 2) EMBEDDINGS

Tokens are converted into dense vectors (numbers) that the model understands.



★ IMPORTANT CONCEPTS

- Vocabulary: Set of all unique tokens the model knows.
- Token ID: Unique number assigned to each token.
- Embedding: Numerical representation of a token.

★ WHY IT MATTERS

- Models don't understand text.
- They understand numbers.
- Tokenization + Embeddings turn text into something models can process.



TEXT → TOKENS → VECTORS → LLM

★ REMEMBER



Before the model can think,
it must convert text → tokens → vectors.

– Ketan Sagare

POSITIONAL INFORMATION & RoPE

★ WHY POSITION MATTERS

Transformers process all tokens in parallel and don't know the order by default.

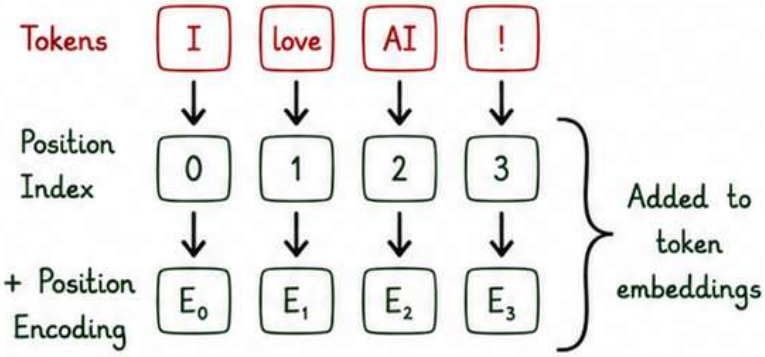
Example:

"Dog bites man" ≠ "Man bites dog"

Position tells the model where each token appears in the sequence.

★ POSITIONAL INFORMATION

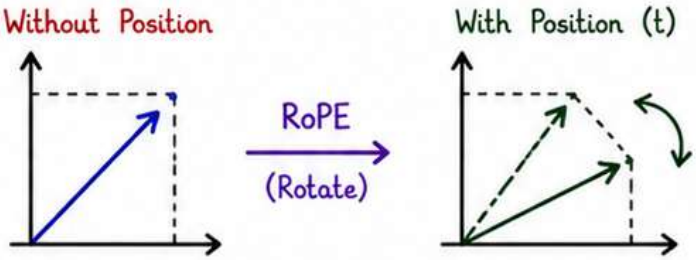
Adds information about the position of each token to its representation.



★ WHAT IS RoPE?

RoPE (Rotary Positional Embedding) encodes position by rotating the token representation in a multi-dimensional space.

2D VISUAL EXAMPLE



Different positions = Different rotation angles

★ WHY RoPE?

- ✓ Captures relative positions effectively
- ✓ Works well with long sequences
- ✓ Better generalization to longer contexts
- ✓ No extra parameters (efficient)

★ COMMON IN MODERN LLMs

Used in models like:

Llama | Mistral | Gemma

and many more.

★ REMEMBER



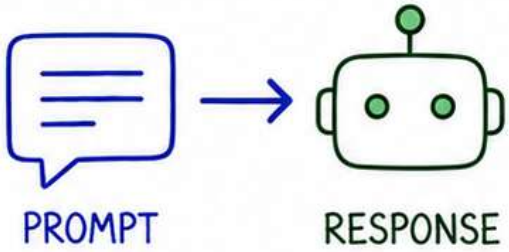
Attention tells WHAT relates,
Position tells WHERE it occurs.

– Ketan Sagare

LLM INFERENCE: PREFILL & DECODE

★ OVERVIEW

Inference is the process of using a trained LLM to generate a response for a given prompt.



★ TWO MAIN STAGES

1) PREFILL
- Process the entire input prompt.
- Compute the initial attention states.

→

2) DECODE
- Generate output tokens one by one.
- Each new token uses previous context.

→

RESPONSE
Generated output tokens
□ □ □ ... □

★ HOW IT WORKS (FLOW)

INPUT PROMPT "Explain AI." → PREFILL (Read & Understand the prompt) → DECODE (Generate tokens one by one) → OUTPUT "AI is the field of creating ..."

★ KEY POINTS

PREFILL
- Compute-heavy
- Parallelizable
- Done once per request

DECODE
- Latency-sensitive
- Mostly sequential
- Repeats until end token

TIP
Good systems optimize both prefill speed and decode latency.

★ REMEMBER

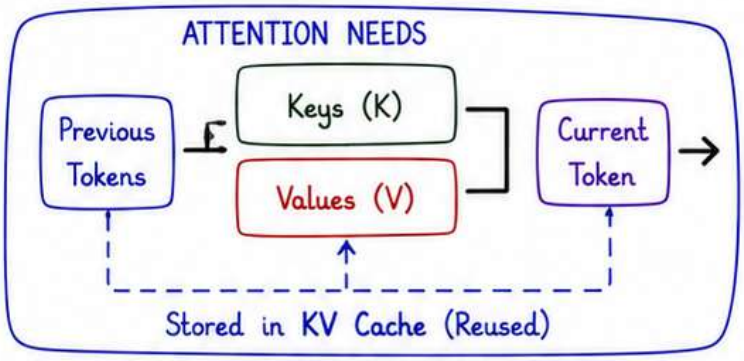
Prefill reads the prompt. Decode generates the answer.

– Ketan Sagare

KV CACHE (KEY-VALUE CACHE)

★ OVERVIEW

During generation, the model needs previous attention results again and again. KV Cache stores them so we don't recompute.



★ HOW IT WORKS

1. PREFILL
Process the entire prompt once.

→

2. STORE
Store Keys (K) and Values (V) in cache.

→

3. DECODE
For each new token, reuse cached K, V from previous tokens.

→

4. REPEAT
Add new K, V to cache and continue.

★ BENEFITS

- ✓ Much faster generation
- ✓ Lower compute (no recomputation)
- ✓ Lower latency
- ✓ Better throughput

★ TRADE-OFF

⚠ More cache → More GPU memory

Larger context = Bigger cache = More memory usage

★ WHAT IS STORED?

KEYS (K)
Represent what each token contains.

VALUES (V)
Represent the actual information (content).

Together K + V
help compute attention for new tokens efficiently.

★ REMEMBER

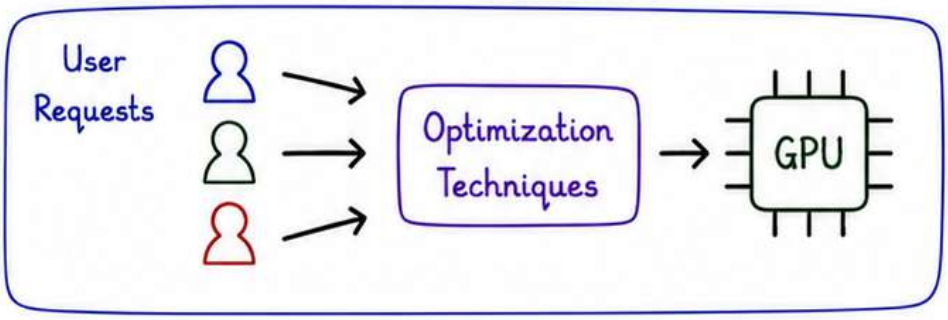
KV Cache trades memory for speed.
Store once, reuse many times!

– Ketan Sagare

BATCHING & INFERENCE OPTIMIZATION

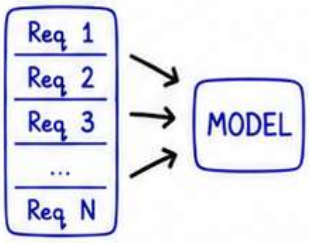
★ OVERVIEW

Optimization techniques help us serve MORE requests with LESS cost and latency.



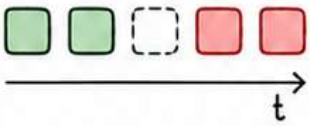
★ KEY TECHNIQUES

1. BATCHING



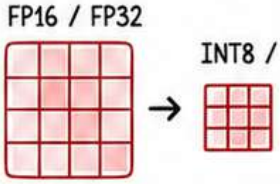
- Process multiple requests together.

2. CONTINUOUS BATCHING



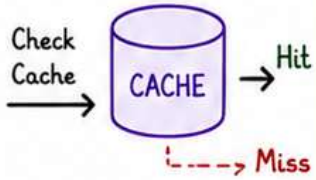
- Add new requests while others are generating.

3. QUANTIZATION



- Use lower precision weights to save memory & speed up.

4. CACHING



- Reuse previous results (KV cache, embeddings, etc.).

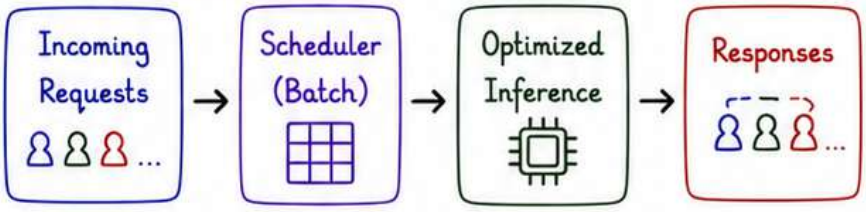
★ GOAL

Higher Throughput
Serve more requests per second.

Lower Latency
Reduce time to first token and per token.

Lower Cost
Use compute resources more efficiently.

★ SIMPLE FLOW



Smart batching + caching + quantization = Faster, Cheaper, Scalable inference.

★ REMEMBER

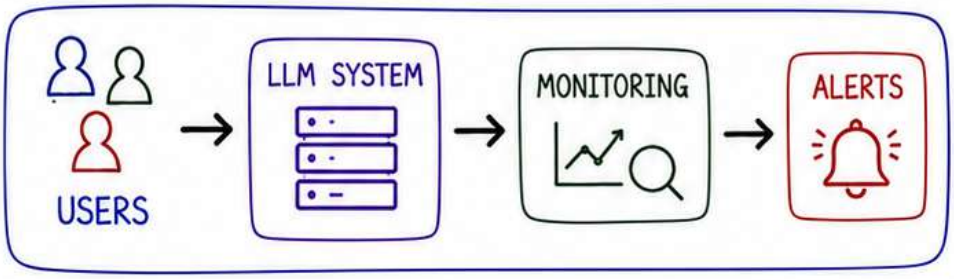
Optimize to serve MORE users, with LESS cost and latency.

– Ketan Sagare

MONITORING & RELIABILITY

★ OVERVIEW

Monitoring helps us maintain healthy, reliable and high-performing LLM inference systems.



★ KEY METRICS TO MONITOR

LATENCY
- Time to first token
- Total response time

THROUGHPUT
- Requests per second
- Tokens per second

ERROR RATE
- Failed requests
- Timeout / Exception rate

GPU UTILIZATION
- GPU usage %
- Memory usage %

QUEUE SIZE
- Waiting requests
- Backlog size

★ BEST PRACTICES

- ✓ Set SLOs for latency, availability & errors
- ✓ Use dashboards for real-time visibility
- ✓ Enable logging & tracing for requests
- ✓ Set alerts for anomalies & thresholds
- ✓ Continuously analyze & improve

★ ALERT EXAMPLES

High Latency	Alert if P95 latency > threshold (e.g. 3s)
High Error Rate	Alert if error rate > 1% for 5 min
High GPU Memory	Alert if GPU memory usage > 90%

★ COMMON TOOLS

PROMETHEUS — Metrics collection & monitoring

GRAFANA — Dashboards & visualization

LOKI — Log aggregation & analysis

JAEGER — Distributed tracing

ALERTMANAGER — Alerts routing & notifications

★ REMEMBER

You can't improve what you don't measure.

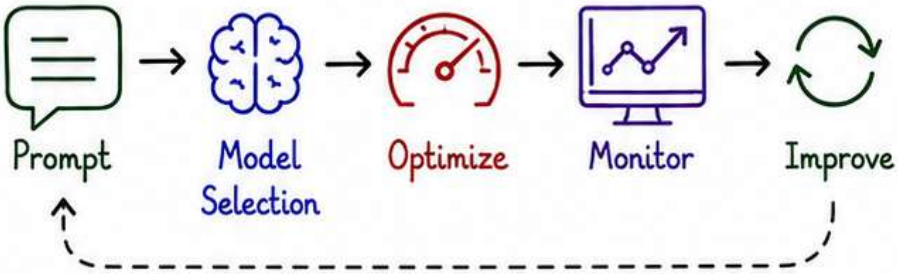
Monitor → Alert → Analyze → Improve.

– Ketan Sagare

BEST PRACTICES & NEXT STEPS

★ 1. BEST PRACTICES

- ☑ Keep prompts clear, focused & concise
- ☑ Use the right model for the right task
- ☑ Optimize for latency, cost & quality
- ☑ Monitor, evaluate & improve continuously
- ☑ Handle errors, timeouts & edge cases



★ CHALLENGES IN INFERENCE

- High Latency & timeouts
- High cost at scale
- Hallucinations & bad outputs
- Rate limits, failures, retries

★ NEXT STEPS FOR YOU



★ REMEMBER

Great inference is the result of the right model, smart design, continuous monitoring & constant improvement.

Keep Learning. Keep Building. Keep Improving.
That's how you build impact with LLMs!

– Ketan Sagare